

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE CODE: CSA1402 COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: *Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 20

Total Marks:100

Annexure A

SHORT ANSWER TEST

Code of Conduct:

*I Athavan K (192524036) (Name / Reg No) certify that this submission is my original work
and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Athavan K

Faculty Incharge

SHORT ANSWER TEST

1. Construct language for the following Regular Expression:2

$(a|b)^*a(a|b)|(a|b)$

2. a) Illustrate the predictive parser for the following grammar. 4

$S \rightarrow (L) | a$

$L \rightarrow L, S | S$

3. Analyse, is it possible, by modifying the grammar in any way to construct

predictive parser for the language of

$S \rightarrow SS+ | SS^* | a$

String: "aa+a" 4

4. Eliminate left recursion from the grammar 2

5. The following grammar contains common prefixes, making it unsuitable for predictive parsing.

$A \rightarrow abcD | abcE | f$

Apply left factoring to remove the common prefixes and rewrite the grammar.5

6. Consider the following statement:10

$Degree - F = Degree - C * 1.8 + 32$

Show the representation of this statement after each phase of the compiler and explain.

7. Consider the following C code fragment:4

`sum = 0;`

`for (i = 1; i < 5; i++)`

`sum = sum + i;`

Show the representation of the above code after each phase of the compiler.2

8. Consider the following grammar.

$S \rightarrow aAb, A \rightarrow c| \epsilon, B \rightarrow d| \epsilon$

Calculate FIRST and FOLLOW, Obtain the Predictive Parsing Table.

not. Show the parser moves for the input string **acdb**.

9. Write a simple Lex program to recognize and count identifiers in a given input.2

10. Write the rule to Eliminate Left Recursion and Left Factoring.2

11. What are the actions performed by a shift-reduce parser? Explain each action with

a suitable example.3

12. Consider the following grammar:

$S \rightarrow BB^*B | r$

Find the leftmost derivation and construct the parse tree for the input string

pqr.3

13. Consider Grammar: $E \rightarrow E+T \mid T$ $T \rightarrow T^*F \mid F$ $F \rightarrow (E) \mid id$

Eliminate left recursion and find FIRST and FOLLOW 5

14. **For the following grammar, determine whether the string abcc is accepted**

using shift-reduce parsing. Show all shift and reduce operations.

10

$$S \rightarrow ABA \rightarrow aA|aB \rightarrow dB|c$$

15. For the following grammar, verify whether the string **abbc** is valid using

shift-reduce parsing. 10

$$S \rightarrow ABA \rightarrow bA|bB \rightarrow c$$

Show the complete sequence of stack, input, and parser actions.

16. Consider the following grammar: 10

$$S' \rightarrow SS \rightarrow aABA \rightarrow bA|dB \rightarrow d$$

- Construct the canonical collection of LR(0) items.

- Construct the SLR(1) parsing table.

17. Consider the following grammar.

$$S \rightarrow ESS' | a, S' \rightarrow \epsilon S | \epsilon, E \rightarrow b$$

Obtain the Predictive Parsing Table. 5

18. Do left factoring in the following grammar: 3

$$A \rightarrow aAB|aA|a$$

$$B \rightarrow bB|b$$

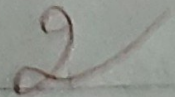
19. Construct the LL(1) Parsing Table for the following grammars. 7

$$S \rightarrow ABCA \rightarrow a|\epsilon B \rightarrow r|\epsilon C \rightarrow b|\epsilon$$

20. Construct the LL(1) Parsing Table for the following grammars. 7

$$S \rightarrow 1S\#|qABCA \rightarrow a|bbDB \rightarrow a|\epsilon C \rightarrow b|\epsilon D \rightarrow c|\epsilon$$

~~1000~~
1000



75/100

$$\text{First}(L) = \{a\}$$
$$\text{Follow}(C_3) = \{4, 3\}$$

~~FOLLOW C₁~~ = { ϕ }

id C O e

5 9207

2

52 1

Stack Implementation:

Stock

eight

② 4/21/21

§ 5

00 f e

5 2 0 8 1

$S \rightarrow SS^+$

9081

 $Be + e$
$$s' \rightarrow s + s$$
 $s \rightarrow ad$ ~~5051 + 5~~

0045

3-7 0 31

$$3 \rightarrow s \overset{+}{s} s \mid s \overset{x}{s} s \mid e$$

$q + s' a os'$

 $00 + 2$

$s \rightarrow s^*$

970

Exhibit

517

First (5) $\rightarrow$ 203

$$\text{First}(s') \rightarrow \{c\}$$

Follow (5) $\rightarrow$

$$g + s' s'$$

42

$$S S' + S''$$

12

31

70

Suppose,

Semantic analysis:

allows numerical constants

The given grammar is incorrect, $S \rightarrow SB | SB | C$ is ~~incorrect~~

$S \rightarrow SB | SB | C$

$B \rightarrow bS' | \epsilon$

$S' \rightarrow aS' | bS' | \epsilon$

$A \rightarrow abCD | abCE | f$

$FOLLOW(A) \rightarrow \{ \overset{a}{\cancel{ab}}, b \}$

$FOLLOW(A) \rightarrow \{ \cancel{f}, b \}$

$$Degree F = Degree - C \times 1.8 + 32$$

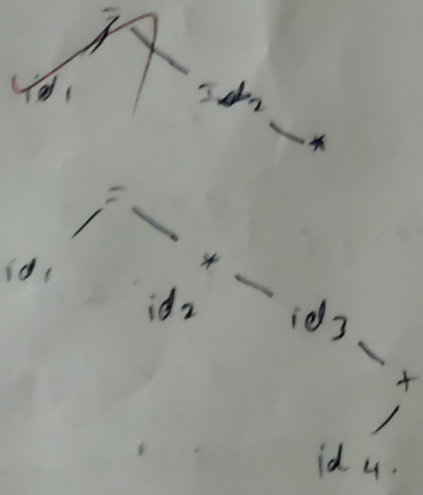
Answer:

$$Degree F = Degree - C \times 1.8 + 32$$

$$21017 \Rightarrow 21027 \Rightarrow 217 \Rightarrow 21037 \Rightarrow 2172 \Rightarrow 21047$$

Syntax:

Tree:



7

sum = 0;

for (i = 1; i < 5; i++)

sum = sum + i

semantic analysis: converts numerical as into integers

Intermediate code generator:

temp1 = id3 + id4

temp2 = id2 * temp1

id1 = temp2

code optimizer:

temp1 = id3 + id4

id1 = id2 * temp1

code generator:

mov id3, r1

mov id

code:

mov id4, r1

mov id3, r2

~~ADD~~
~~mov~~ r2, r1

mul id2, r1

mov r1, id1

7

sum = 0;

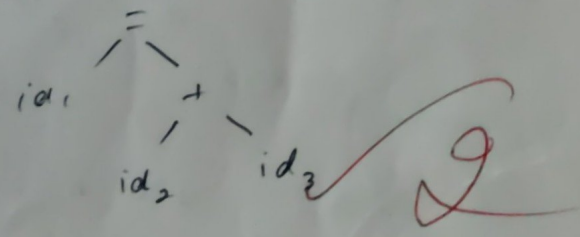
for (i = 1; i < 5; i++)

lexical:

sum = sum + i;

id1 = id2 + id3

syntact: syntor:



semantic analysis:

converts numerical into integers but here no whole numbers.

Intermediate code generator:

temp1 = id2 + id3

id1 = temp1

* $id_1 = id_2 + id_3$

Code :

mov id_3, r_1

mov id_2, r_2

add r_1, r_2

mov r_2, id_4

$S \rightarrow CABH, A \rightarrow C1A, B \rightarrow d1A$

First(S) $\rightarrow \{C\}$

First(A) $\rightarrow \{C, 1\}$

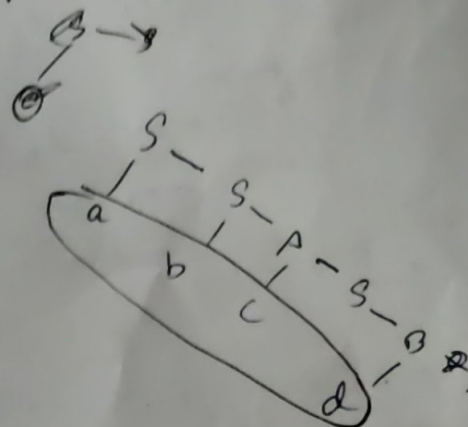
First(B) $\rightarrow \{d, 1\}$

Follow(S) $\rightarrow \{\$, \}$

Follow(A) $\rightarrow \{d, \$\}$

Follow(B) $\rightarrow \{b, \$\}$

Input string : abcd



id a b c d ϵ

$S \rightarrow CABb$

$A \rightarrow C$ $A \rightarrow \epsilon$ $A \rightarrow \epsilon$

$B \rightarrow d$ $B \rightarrow \epsilon$

$B \rightarrow \epsilon$

S

A

B

10 Left Recursion:

* The ^{the grammar} first ~~id~~ to be the

$$A \rightarrow \alpha A \mid \beta$$

* Then it reduce of A^0 ,

$$A \rightarrow \beta A'$$

* If the non terminal has same as the production then non terminal then it should be performed left recursion

* Ex) $A \rightarrow \bar{A} + B \mid 0$ ^{some non terminal}, here left recursion applicable

$E \rightarrow \underline{CE} \mid id$, here not applicable
_{non terminal}

Left Factoring:

* it should follow $N \rightarrow id(tes)$, in this format

* It acts same as left recursion.

Shift-Reduce:

* In a production if the element which is ^{greater} ~~less~~ than the up next element then it should shifted.

2 Ex) $id > (\mid) > * \mid / > + \mid - > \$$

* In a production, if the first element is less lesser priority than the up next element, then it should be reduced

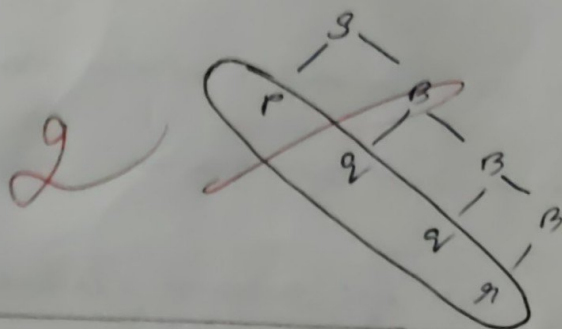
The priority order:

id > c / > + / - > * / - > \$

3 -> P B

B -> 2 B 1 2

+ 0 first :: P 2 2 2



2

1.

include <iostream>

void main()

{

for if (id = +, -, *, /)

count++;

else

count++;

}

Print (count) " " id " " count;

1.

}

Output:

3 = A + B + C.

no of identifier : 2.

E -> E + T

T -> T *

T -> T /

left

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

Left Recursion:

$$E \rightarrow E + T \mid T$$

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \epsilon$$

$$T \rightarrow T * F \mid F$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

$$\text{First}(E) \rightarrow \{ (, \epsilon \}$$

$$\text{First}(E') \rightarrow \{ +, \epsilon \}$$

$$\text{First}(T) \rightarrow \{ (, \epsilon \}$$

$$\text{First}(T') \rightarrow \{ *, \epsilon \}$$

$$\text{First}(F) \rightarrow \{ (, id \}$$

$$\text{Follow}(E) \rightarrow \{ \$, \epsilon \}$$

$$\text{Follow}(E') \rightarrow \{ \$, \epsilon \}$$

$$\text{Follow}(T) \rightarrow \{ +, \epsilon, \$ \}$$

$$\text{Follow}(T') \rightarrow \{ *, +, \epsilon, \$ \}$$

$$\text{Follow}(F) \rightarrow \{ +, *, \epsilon, \$ \}$$

	id	++	++	c	>	\$
E						E → c
E'				E' → TE'		E' → TE
T						T → TE
T'				T' → TE'		T' → TE
F				F → id		F → (E) → (E)

14

$S \rightarrow AB$

$A \rightarrow aA|a$

$B \rightarrow cB|c$

abcc

so the given input is not correct grammar

stack

input

output

\$

abcc\$

shift [$B \rightarrow aB$]

\$AB

abcc\$

reduce [$A \rightarrow a$]

\$aB

abcc\$

stack

~~Input~~

output:

\$S

abcc

shift [$S \rightarrow AB$]

\$AB

abcc

reduce [$A \rightarrow a$]

\$aB

abcc

reduce [$B \rightarrow c$]

\$B

bcc

reduce [$B \rightarrow c$]

\$CB

bcc

shift [b]

\$CB

cc

Stack	input	Output	
\$ S	abbc \$	reduce [B → c]	S → CAB
\$ ^a AB	abbc \$		A → bA b
\$ AB	bbc \$	reduce (A → bA)	B → c
\$ bAB	bbc \$		
\$ AB	bc \$	reduce [A → b]	
\$ bB	bc \$	reduce [B → c]	
\$ c	\$		
\$	\$		

$$S' \rightarrow S$$

$$S \rightarrow CAB$$

$$A \rightarrow bA | b$$

$$B \rightarrow c$$

$$\text{First}(S) = \{a\}$$

$$\text{First}(S) = \{a\}$$

$$\text{First}(A) = \{b, c\}$$

$$\text{First}(B) = \{c\}$$

Closure:

$$S \rightarrow \cdot S$$

$$S \rightarrow \cdot CAB$$

$$A \rightarrow \cdot bA | \cdot c$$

$$B \rightarrow \cdot c$$

$$\text{Follow}(S') = \{ \$ \}$$

$$\text{Follow}(S) = \{ \$ \}$$

$$\text{Follow}(A) = \{ d, \$ \}$$

$$\text{Follow}(B) = \{ d, \$ \}$$

$$S' \rightarrow S$$

$$S \rightarrow CAB$$

$$A \rightarrow bA$$

$$A \rightarrow c$$

$$B \rightarrow d$$

horo

$$\text{I}_0 (I_0, S) = I_1$$

$$S' \rightarrow S.$$

$$S \rightarrow \cdot CAB$$

$$A \rightarrow \cdot bA | \cdot c$$

$$B \rightarrow \cdot d$$

$$\text{Goto}(\pi_0, \beta) = \pi_2$$

$$S \rightarrow a \cdot AB$$

$$\text{Goto}(\pi_0, A) = \pi_3$$

$$S \rightarrow A \cdot b \cdot A$$

$$A \rightarrow c \cdot$$

$$\text{Goto}(\pi_0, B) = \pi_4$$

$$B \rightarrow d \cdot$$

$$\text{Goto}(\pi_1, \beta') = \pi_1$$

$$\text{Goto}(\pi_1, \beta) = \pi_5$$

$$S \rightarrow a \cdot A \cdot B$$

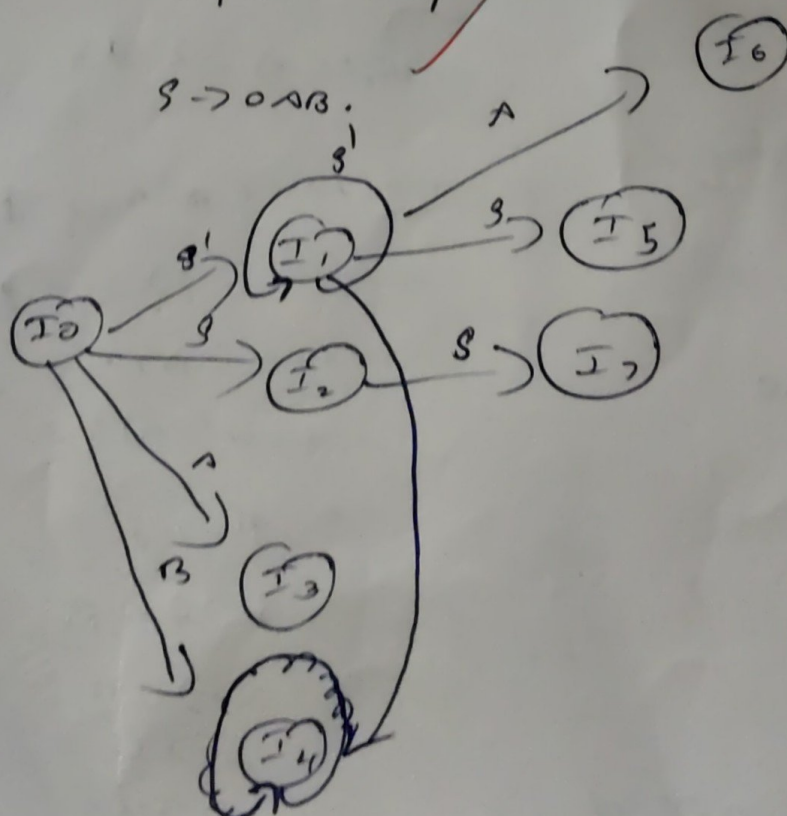
$$\text{Goto}(\pi_1, A) = \pi_6$$

$$A \rightarrow b \cdot A \cdot$$

$$A \rightarrow c \cdot$$

$$\text{Goto}(\pi_1, B) = \pi_4$$

$$\text{Goto}(\pi_2, \beta) = \pi_7$$



01010.

	a	b	c	d	\$	s'	s	A	B
0	1 25 2 25				π_1	1		3	4
1						1	10	6	
2							7		
3					π_4				
4					π_5				
5									
6					π_3	π_3			
7					π_2				

17

$$S \rightarrow iE \text{ or } sc' \text{ or } a$$

$$S' \rightarrow es \text{ or } e$$

$$E \rightarrow b$$

$$\text{First}(S) = \{ b, a \}$$

$$\text{First}(S') = \{ e \}$$

$$\text{First}(E) = \{ b \}$$

$$\text{Follow}(S) = \{ \$ \}$$

$$\text{Follow}(S') = \{ b, e, \$ \}$$

$$\text{Follow}(E) = \{ e, a, \$ \}$$

18 $A \rightarrow a A^1 e A^1 a$

$B \rightarrow b B^1 b$



19

$S \rightarrow ABC$

$A \rightarrow a | \epsilon$

$B \rightarrow a | \epsilon$

$C \rightarrow b | \epsilon$

$\text{First}(S) = \{a\}$

$\text{First}(A) = \{a, \epsilon\}$

$\text{First}(B) = \{a, \epsilon\}$

$\text{First}(C) = \{b, \epsilon\}$

$\text{Follow}(S) = \{\$ \}$

$\text{Follow}(A) = \{a, \$ \}$

$\text{Follow}(B) = \{a, b, \$ \}$

$\text{Follow}(C) = \{b, \$ \}$ $\rightarrow$ Empty set

2

	a	a	b	\$
S	$S \rightarrow A B C$			$S \rightarrow \epsilon A B C$
A	$A \rightarrow a$	$A \rightarrow \epsilon$		$A \rightarrow \epsilon$
B		$B \rightarrow a$	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$
C			$C \rightarrow b$	$C \rightarrow \epsilon$

$$S \rightarrow 1SH \mid 9ABC$$

$$A \rightarrow a \mid b b D$$

$$B \rightarrow 0 \mid \epsilon$$

$$C \rightarrow b \mid \epsilon$$

$$D \rightarrow c \mid \epsilon$$

$$\text{First}(S) = \{1, 9\}$$

$$\text{First}(A) = \{a, b\}$$

$$\text{First}(B) = \{0, \epsilon\}$$

$$\text{First}(C) = \{b, \epsilon\}$$

$$\text{First}(D) = \{c, \epsilon\}$$

$$\text{Follow}(S) = \{\$ \}$$

$$\text{Follow}(A) = \{c, \$ \}$$

$$\text{Follow}(B) = \{c, \$ \}$$

$$\text{Follow}(C) = \{c, \$ \}$$

$$\text{Follow}(D) = \{c, \$ \}$$

RUBRICS FOR CALCULATION

CSA1402 - Compiler Design | CO2 Assessment Tool 2

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand